Campus-DDoS-SDN: A Python-based testbed for multi-vector DDoS attack detection in software-defined campus networks

Anna Olenka Calderón Leiva[a], Ricardo Alexander Llanos Lozano[a], Cristhian Emilio Martínez Casas[a,*] and Alessandro Piero Poves Martínez[a]

[a] *Universidad Nacional del Centro del Perú, , Huancayo, , , Peru*

ARTICLE INFO

Keywords:
Software defined networking
Machine learning
DDoS attack detection
Mininet
Network traffic classification

ABSTRACT

TODO: resumen de 100 palabras. Debe cubrir: que hace el software (entorno reproducible Mininet+Ryu+ML para generar y clasificar trafico DDoS multi-clase), como se diferencia del trabajo de referencia (deteccion de 4 tipos de ataque en vez de solo ICMP flood, entorno reproducible via Docker), y el resultado principal obtenido.

1. Introduction

Software-defined networking (SDN) decouples the control plane from the data plane, concentrating network intelligence in a logically centralized controller [1]. This centralization simplifies network management, but it also turns the controller into a single point of failure: a distributed denial-of-service (DDoS) attack that saturates the controller, the switches, or a hosted service can degrade or take down the entire network segment it governs. Detecting such attacks early, and separating them reliably from legitimate traffic spikes, requires datasets that are both representative of real SDN deployments and precisely labelled.

Most publicly available intrusion-detection datasets, such as NSL-KDD [6], CICIDS2017 [4], and CICDDoS2019 [5], were not captured inside an SDN environment: they lack OpenFlow-specific signals, such as flow-table occupancy or controller load, and are nonetheless often reused out of context to train SDN-targeted classifiers. Recent work has started to close this gap by generating DDoS datasets directly inside Mininet/Ryu testbeds. Kumar and Gupta released two single-vector datasets built this way: a TCP-SYN flood dataset [2] and, more recently, an ICMP-flood dataset [3] in which normal traffic is captured by connecting a Mininet host to a live internet connection and browsing real websites (YouTube, Facebook, news, etc.), instead of relying on synthetic traffic generators. This real-traffic approach increases realism, but it introduces two practical limitations for a reusable benchmark: the resulting dataset is tied to a single attack vector, and it is not reproducible across runs, since the captured content and timing depend on whatever the live internet happens to serve at capture time.

This work, Campus-DDoS-SDN, addresses both limitations within a single, fully reproducible testbed. Four attack vectors (TCP-SYN flood, HTTP flood, Slowloris, and ICMP flood) are launched from already-compromised hosts against one shared victim, a web portal, inside a Mininet/Ryu topology that mimics a university campus: a core/DMZ switch layout plus one emulated laboratory per academic area. Unlike prior single-vector or fully-separated-capture datasets, legitimate users keep browsing the portal throughout every attack, so each capture mixes benign and malicious flows against the same target, as a real DDoS incident would. Ground-truth labels are assigned per source IP rather than per capture file, and every run additionally logs the attack's effect on the victim as perceived by a real client (HTTP response code, latency, active connections), alongside network-level indicators (packets and bytes per second, OpenFlow flow-table size). The complete environment, including the traffic generators, the custom web portal, and the labelling pipeline, is packaged as a Docker image to remove any dependency on a live internet connection or on a specific host operating system.

The main contributions of this work are:

i. A multi-class DDoS dataset covering four attack vectors against a single victim inside a realistic, multi-segment SDN topology, instead of a single attack type against a flat network.

ii. Per-flow, per-source-IP labelling that keeps legitimate and malicious traffic mixed within the same capture, closer to a real attack scenario than the fully-separated captures used in prior work.

iii. Combined network-level (flow and packet statistics via CICFlowMeter) and application-level (HTTP status, latency, connection saturation) impact metrics, captured automatically on every run.

iv. A one-command, fully reproducible Docker environment that does not depend on a live internet connection, addressing the reproducibility limitation of real-traffic-based SDN datasets.

*Corresponding author
cristhianmartinezcasas@gmail.com (C.E.M. Casas)
ORCID(s):

Table 1
Code metadata.

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.0
C2	Permanent link to code/repository used for this code version	TODO: URL del repo
C3	Permanent link to Reproducible Capsule	–
C4	Legal Code License	TODO: elegir licencia (p.ej. MIT)
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python, Bash, Mininet, Open vSwitch, Ryu, CICFlowMeter, Docker
C7	Compilation requirements, operating environments & dependencies	WSL2/Ubuntu 24.04 o Docker; Python 3.9 (Ryu) y Python 3.12 (CICFlowMeter);
C8	Link to developer documentation/manual	Ver `README.md` del repositorio
C9	Support email for questions	TODO: email de contacto

2. Software description

2.1. Architecture overview

Topologia Mininet de 5 hosts (servidor propio, 2 clientes, 2 atacantes) sobre un switch Open vSwitch (datapath userspace) controlado por Ryu; generacion de 4 tipos de ataque (SYN flood, HTTP flood, Slowloris, ICMP flood) capturados por separado; conversion a CSV con CICFlowMeter (y un agrupador de flujos propio para ICMP, que CICFlowMeter no soporta); todo empaquetado en una imagen Docker para reproducibilidad completa. TODO: ampliar.

2.2. Software functionalities

TODO: enumerar los scripts principales (`scripts/mininet`, `scripts/wsl`, `scripts/docker`) y que hace cada uno.

3. Impact overview

TODO: completar. Punto de partida: a diferencia de [3], que depende de una conexion a internet real y por lo tanto no es reproducible entre corridas, este trabajo habilita comparaciones reproducibles de metodos de deteccion multi-clase de DDoS en SDN.

3.1. Ongoing research projects using the software

TODO (si aplica).

3.2. Limitations and future improvements

El trafico normal se genera de forma sintetica y controlada (en vez de navegacion real a internet, como en [3]), lo que reduce la diversidad/realismo del trafico a cambio de reproducibilidad total. Trabajo futuro: deteccion en tiempo real mediante una aplicacion Ryu propia que clasifique estadisticas OpenFlow en vivo.

3.3. List of scholarly publications enabled by this software

TODO (si aplica, una vez publicado).

4. Software and hardware requirements

Acknowledgements

TODO (opcional).

References

[1] Kreutz, D., Ramos, F.M.V., Veríssimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2015. Software-defined networking: A comprehensive survey. Proceedings of the IEEE 103, 14–76. doi:10.1109/JPROC.2014.2371999.

[2] Kumar, S., Gupta, S., 2025. SDN TCP-SYN dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks. Data in Brief 59, 111314. doi:10.1016/j.dib.2025.111314.

[3] Kumar, S., Gupta, S., 2026. ICMP-Flood-SDN: A Python based machine learning application for ICMP flood DDoS attack detection in software defined networks. Software Impacts 27, 100819. doi:10.1016/j.simpa.2026.100819.

[4] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116. doi:10.5220/0006639801080116.

[5] Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A., 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: Proceedings of the 53rd IEEE International Carnahan Conference on Security Technology (ICCST), pp. 1–8. doi:10.1109/CCST.2019.8888419.

[6] Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the KDD CUP 99 data set, in: Proceedings of the 2nd IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA).

Table 2
Testbed setup requirements.

N.°	Descripcion	Herramienta	Version
1	Emulador de red	Mininet	TODO
2	Controlador SDN	Ryu	4.34
3	SYN / ICMP flood	hping3	TODO
4	Ataque de baja tasa	slowhttptest	TODO
5	pcap a CSV	CICFlowMeter	0.5.0